DESIGN AND ANALYSIS OF ALGORITHMS

Dijkstra’s Shortest Path Algorithm

A greedy algorithm for finding shortest paths

Aditya Koshta | PRN: 25070521064

Generated using ChatGPT and Canva AI

1. What is Dijkstra’s Algorithm?

Finding the least-cost route through a weighted graph

Definition

Dijkstra’s algorithm finds the shortest distance from one source vertex to every other vertex in a graph with non-negative edge weights.

Key idea

It repeatedly chooses the unvisited vertex with the smallest known distance and updates distances to its neighbours.

Important terms

- Vertex: a node in the graph
- Edge weight: cost or distance between nodes
- Source: the starting vertex
- Greedy choice: smallest temporary distance

2. How does it work?

The algorithm improves distance estimates step by step

1 Initialize

Source distance = 0; all others = infinity.

2 Select

Choose the unvisited vertex with minimum distance.

3 Relax edges

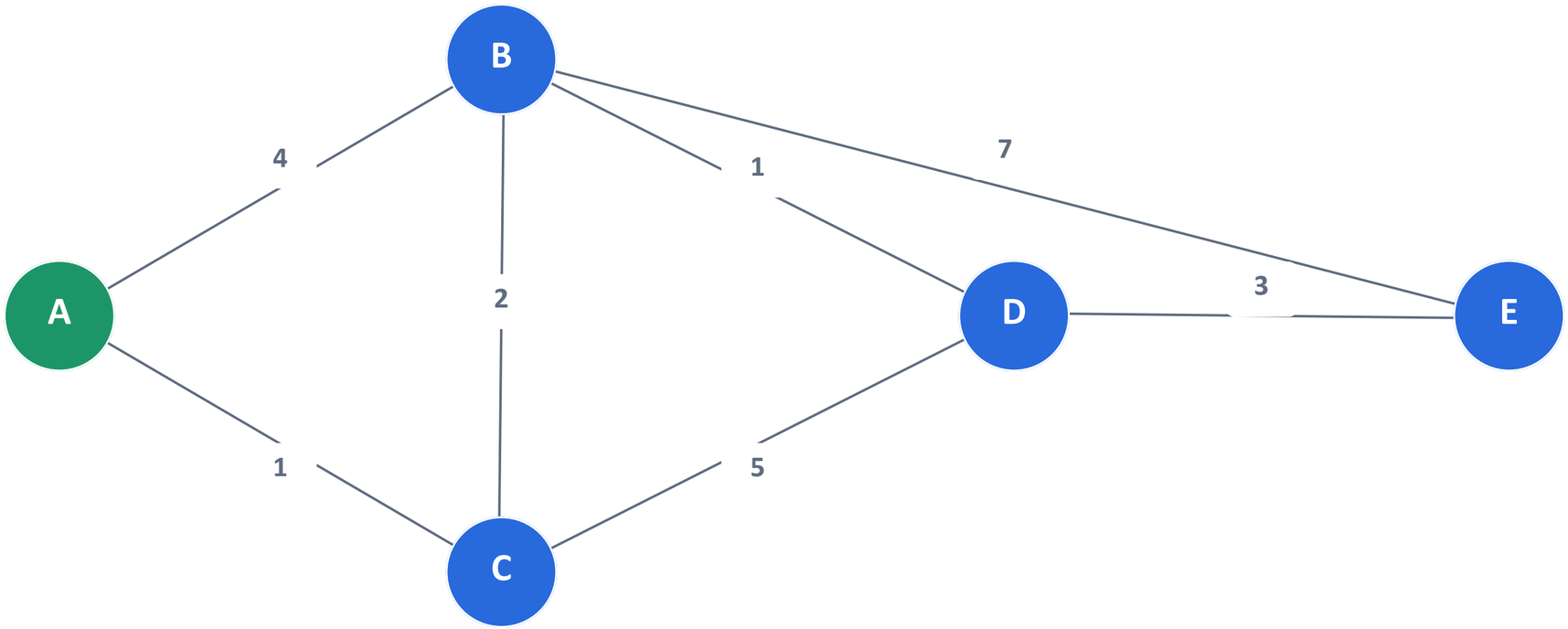
If going through this vertex gives a shorter route, update the neighbour.

4 Repeat

Mark the vertex visited and continue until all reachable vertices are processed.

3. Worked example: weighted graph

Find shortest distances starting from vertex A



Edge labels represent weights (costs). All weights are non-negative.

4. Solving the example

Start at A and repeatedly update the best known distances

Vertex	Shortest distance from A	One shortest path
A	0	A
C	1	A → C
B	3	A → C → B
D	4	A → C → B → D
E	7	A → C → B → D → E

Example update: dist(B) = min(4, dist(C) + 2) = min(4, 1 + 2) = 3.

5. Pseudocode

Simple version using a distance array and visited set

```
DIJKSTRA(Graph, source):
  for each vertex v:
    dist[v] = INFINITY
    visited[v] = false

  dist[source] = 0

  repeat |V| times:
    u = unvisited vertex with minimum dist[u]
    if dist[u] == INFINITY:
      break
    visited[u] = true

    for each neighbour v of u:
      if not visited[v] and
        dist[u] + weight(u, v) < dist[v]:
         dist[v] = dist[u] + weight(u, v)

  return dist
```

Input

Weighted graph and source vertex

Output

Shortest distance from source to each reachable vertex

6. Time complexity and limitations

Performance depends on the graph representation

Array / adjacency matrix

Time: $O(V^2)$
Simple to implement; suitable for dense or small graphs.

Binary heap + adjacency list

Time: $O((V + E) \log V)$
Often efficient for sparse graphs.

Space complexity

Typically $O(V + E)$ with an adjacency list.

Limitation

Does not work correctly with negative-weight edges.
Consider Bellman–Ford for those graphs.

7. Applications and conclusion

- GPS and route planning
- Computer-network routing
- Robotics and game pathfinding

Conclusion: Dijkstra’s algorithm uses a greedy strategy to find shortest paths efficiently when edge weights are non-negative.

Generated using ChatGPT and Canva AI

Aditya Koshta | PRN: 25070521064